

Reviewer Pack — Minimal Rank of Symplectic Leaves in Poisson Geometry vs DPL...

Entry 41ae315462ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 21:19:17 UTC

Minimal Rank of Symplectic Leaves in Poisson Geometry vs DPLL Proof Length

Entry ID: 41ae315462ac **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 21:19:17 UTC

1. Conjecture

Field A (mathematical branch): Poisson Geometry **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘For any homogeneous polynomial f defining a symplectic leaf in the Poisson variety associated with an n -vertex graph G , the minimal rank of the matrix representing the closure of the image of f is at least as large as the DPLL proof length for G .’, ‘sentence_2’: ‘This holds for all graphs G with maximum degree Δ and number of vertices at most 40, where the minimal rank is calculated using the standard Gram-Schmidt process over the real numbers.’, ‘sentence_3’: ‘For any graph G that can be refuted by a DPLL proof of length k , there exists a polynomial f such that the rank of its image in Poisson geometry is at least k .’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Poisson geometry provides a natural setting to study symplectic structures on graphs, which could reveal inherent complexity in their structure.’, ‘sentence_2’: ‘The conjecture bridges the gap between geometric structures and proof length, potentially offering new insights into resolution proof complexity.’, ‘sentence_3’: ‘Symplectic leaves are known for their rigidity, and their ranks might correlate with computational hardness as measured by proof lengths.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b7a220bc2bc12af9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all graphs G with maximum degree Δ and number of vertices at most 40, the minimal rank of the matrix representing the closure of the image of f using the Gram-Schmidt process is greater than or equal to the DPLL proof length

for G over 30 random seeds. The conjecture is falsified if any graph G has a minimal rank less than its DPLL proof length.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_graph(n, max_degree):
    graph = [[] for _ in range(n)]
    degrees = [0] * n
    edges_added = 0

    while edges_added < min(n - 1, max_degree * (n - 1) // 2):
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)

        if u != v and v not in graph[u] and degrees[u] < max_degree and degrees[v] < max_degree:
            graph[u].append(v)
            graph[v].append(u)
            degrees[u] += 1
            degrees[v] += 1
            edges_added += 1

    return graph

def dpll_length(graph):
    n = len(graph)

    def dfs(node, parent, path):
        if node in path:
            return len(path) - path.index(node)

        path.append(node)
```

```

min_length = float('inf')

for neighbor in graph[node]:
    if neighbor != parent:
        length = dfs(neighbor, node, path)
        if length < min_length:
            min_length = length

path.pop()
return min_length

max_length = 0
for i in range(n):
    length = dfs(i, -1, [])
    if length > max_length:
        max_length = length

return max_length

def gaussian_elimination(A):
    n = len(A)
    m = len(A[0])

    def swap_rows(A, i, j):
        A[i], A[j] = A[j], A[i]

    def scale_row(A, i, factor):
        for j in range(m):
            A[i][j] *= factor

    def add_multiple_of_row(A, i, j, factor):
        for k in range(m):
            A[i][k] += factor * A[j][k]

    def find_pivot(A, row):
        for col in range(m):
            if A[row][col] != 0:
                return col
        return -1

    def forward_elimination(A):
        i = 0
        j = 0

        while i < n and j < m:
            pivot_col = find_pivot(A, i)

            if pivot_col == -1:
                i += 1
                continue

            swap_rows(A, i, j)
            scale_row(A, i, Fraction(1, A[i][pivot_col]))

            for k in range(n):
                if k != i and A[k][pivot_col] != 0:

```

```

        add_multiple_of_row(A, k, i, -A[k][pivot_col])

        i += 1
        j += 1

    forward_elimination(A)

    rank = sum(1 for row in A if any(x != 0 for x in row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    max_degree = min(n - 1, 3)
    graph = generate_graph(n, max_degree)

    dpll_len = dpll_length(graph)

    if dpll_len == 0:
        return {
            "metric_name": "Minimal Rank",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

    # Construct the polynomial f
    A = [[0] * n for _ in range(n)]
    for u in range(n):
        for v in graph[u]:
            A[u][v] += 1

    rank = gaussian_elimination(A)

    return {
        "metric_name": "Minimal Rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank >= dpll_len,
        "counterexample": f"Graph with DPLL length {dpll_len} and rank {rank}" if rank < dpll_len
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Graph with DPLL length {result['counterexample_length']}\"")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ccaae299.py", line 150, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ccaae299.py", line 133, in run_trial
    rank = gaussian_elimination(A)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ccaae299.py", line 105, in gaussian_elimination
    forward_elimination(A)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ccaae299.py", line 96, in forward_elimination
    scale_row(A, i, Fraction(1, A[i][pivot_col]))
                        ~~~~~^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with proper error handling to ensure it does not crash and then re-evaluate the results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13275
2	preregistration	ollama_remote	glm4:latest	0	0	5699
3	novelty	ollama_remote	glm4:latest	0	0	4832
4	novelty	ollama_remote	glm4:latest	0	0	5289
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15015
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9558
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10613
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14221
9	judge	ollama_remote	glm4:latest	0	0	11093

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89595 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/41ae315462ac.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/41ae315462ac.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/41ae315462ac.tar.gz* (if generated)